

— TWO POINTERS —

Date / /
Intimate

- ✓01. Reverse Vowels of a String
- ✓02. Remove Duplicates from Sorted Array
- ✓03. Reverse words in a String
- ✓04. No. of Subsequences that Satisfy the Given Sum Condition
- ✓05. Minimize maximum pair sum in Array
- ✓06. Count the no. of Good partitions
- ✓07. Minimum length of String After Deleting Similar Ends
- ✓08. Maximum Width Ramp
- ✓09. Partition Labels
- ✓10. Push Dominoes
- ✓11. Find the lexicographically largest String from the Box I
- ✓12. K-diff Pairs in an Array
- ✓13. Compare Version Numbers
14. Count the no. of Fair Pairs

Patterns

- # Opposite Ends
- # Same Direction / Sliding
- # Fast & Slow pointers
- # Two arrays / Two sequences
- # Pointers maintain regions
- # Multiple pointers

01. Reverse Vowels of a String -

Input: $S = \text{"IceCream"}$

Output :- "Acc.Cre.Im"

0 1 2 3 4

'h'	'e'	'l'	'l'	'o'
-----	-----	-----	-----	-----

i j

① y vowel $\rightarrow$ y ? y y + +
 y vowel $\rightarrow$ y ? y keep y pointers as it was

'h'	'e'	'l'	'l'	'o'
0	1	2	3	4

4
9

```

②      if vowel == yes } swap them
      if vowel == yes } i++
                        j++

```

'h'	'o'	'l'	'l'	'e'
0	1	2	3	4

③

u	ɪ	vowel	no	ɛ	} crossed each other
ɪ	j	vowel	no	j	

④

'h'	'o'	'l'	'l'	'e'
0	1	2	3	4

5 4

Stop

Victims

//Code:-

```

public String reverseVowels (String s) {
    int n = s.length();
    char arr[] = s.toCharArray();

    int left = 0;
    int right = n-1;

    while (left < right) {
        if (!isVowel (arr[left])) {
            left++;
        }
        else if (!isVowel (arr[right])) {
            right--;
        }
        else {
            char temp = arr[left];
            arr[left] = arr[right];
            arr[right] = temp;
            left++;
            right--;
        }
    }

    return new String (arr);
}

private boolean isVowel (char ch) {
    if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o'
        || ch == 'u' || ch == 'A' || ch == 'E' || ch == 'I'
        || ch == 'O' || ch == 'U') {
        return true;
    }

    return false;
}

```

TC - $O(N)$ SC - $O(1)$

02. Remove Duplicates from Sorted Array -

Input :- nums = [0, 0, 1, 1, 1, 2, 2, 3, 3, 4]

Output :- 5

DRY RUN :-
 $i =$ unique element
 $j =$ unique element

①

0	0	1	1	1	2	2	3	3	4
---	---	---	---	---	---	---	---	---	---

i j

$i == j$ $j++$

②

0	0	1	1	1	2	2	3	3	4
---	---	---	---	---	---	---	---	---	---

i j

$i != j$ $arr[i] = arr[j]$
 $i++$
 $j++$

③

0	1	1	1	1	2	2	3	3	3	4
---	---	---	---	---	---	---	---	---	---	---

i j

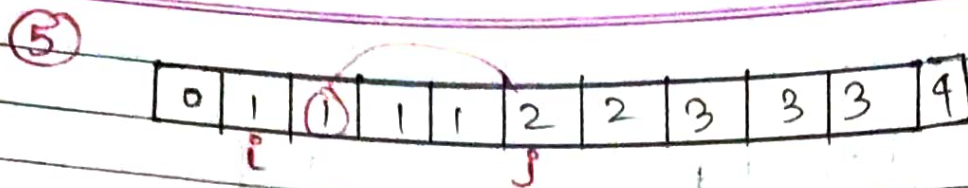
$i == j$ $j++$

④

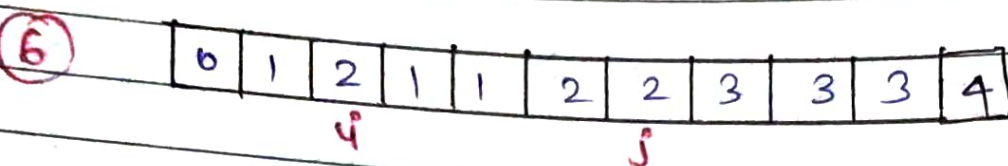
0	1	1	1	1	2	2	3	3	3	4
---	---	---	---	---	---	---	---	---	---	---

i j

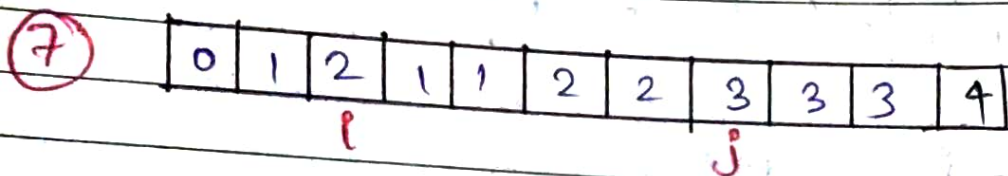
$i == j$ $j++$



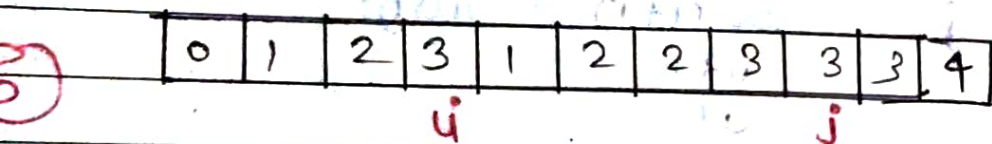
$i \neq j$ $arr[i+1] = arr[j]$
 $i++$, $j++$



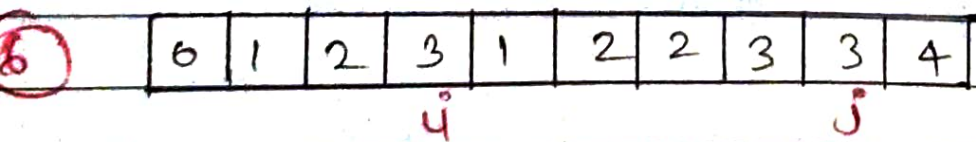
$i == j$ $j++$



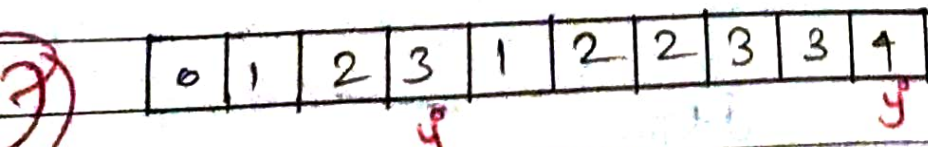
a $i \neq j$ $arr[i+1] = arr[j]$
 $i++$ $j++$



$i == j$ $j++$



$i == j$ $j++$



$i \neq j$ $arr[i+1] = arr[j]$
 $i++$ $j++$

8

0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	2	2	3	3	4

4 9

out of bound

return i+1

Code:-

```
public int removeDuplicates (int[] nums) {
    int n = nums.length;
    int j = 0;

    for (int i=1; i<n; i++) {
        if (nums[i] != nums[j]) {
            nums[j+1] = nums[i];
            j++;
        }
    }

    return j+1;
}
```

Tc - $O(2N)$
Sc - $O(1)$

03. Reverse words in a string -

Input:- " hello world "

Output:- " world hello "

DRY RUN:-

- ① Trim and Replace multiple spaces with a single space.

" hello world "

leading trailing space
will remove

- ② Reverse the Entire String

" dlrow olleh "

- ③ Reverse Each word

" world hello "

" world hello "

//Code:-

```
public String reverseWords(String s) {
    s = s.trim().replaceAll("\\s+", " "); // O(N)
```

```
    char[] arr = s.toCharArray(); // O(N)
    reverse(arr, 0, arr.length - 1); // O(N)
```

```
    int start = 0;
```

```
    for (int i = 0; i <= arr.length; i++) { // O(N)
```

```
        if (i == arr.length || arr[i] == ' ') {
```

```
            reverse(arr, start, i - 1); // O(N)
```

```
            start = i + 1;
```

```
        }
```

```
    }
```

```
    return new String(arr);
```

```
private void reverse(char[] arr, int left, int right) {
```

```
    while (left < right) {
```

```
        char temp = arr[left];
```

```
        arr[left++] = arr[right];
```

```
        arr[right--] = temp;
```

```
    }
```

```
}
```

TC - $O(N) + O(N) + O(N) + O(N) \approx O(N)$

SC - $O(N) + O(N) \approx O(N)$

04. Number of Subsequences That Satisfy the Given Sum Condition- {Amazon}

Input :- nums = ~~[3, 4]~~ [3, 5, 6, 7]

target = 9

Output :- 4

Explanation -

- [3] $\rightarrow \min\{3\} + \max\{3\} \Rightarrow 9$
- [3, 5] $\rightarrow \min\{3\} + \max\{5\} \Rightarrow 8 \leq 9$
- [3, 5, 6] $\rightarrow \min\{3\} + \max\{6\} \Rightarrow 9 \leq 9$
- [3, 6] $\rightarrow \min\{3\} + \max\{6\} \Rightarrow 9 \leq 9$

Dry Run

• Sort - Why sort? because when you sort.

[3, 6, 5, 7] target = 9
3+7 $\Rightarrow 10 > 9$

[3, 6, 5, 7]
3+5 = 8 ≤ 9 (valid)

You can identify easily that these subarray will be valid

[3, 5, 6, 7] 3+6 $\Rightarrow 9 \leq 9$

अब 6 से पहले के सभी elements valid होंगे 3 के लिए क्योंकि इसे min & max से मानकर है और वो (3, ..., 6) ही होंगे

sorted.

(19) { 2, 3, 3, 4, 6, 7 }

$$\begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \\ \wedge & \wedge & \wedge & \wedge & \wedge & \times \\ 2 & 3 & 3 & 4 & 6 & 7 \end{array}$$

$$2 + 7 = 9 < 12$$

target = 12

$$\begin{array}{c} \text{r-l} \\ (0-5) \Rightarrow 5 \end{array}$$

$2^5 \Rightarrow \text{Ans}$

4 को 2 fix रखने पर 2⁵ subarray valid होत

(20) { 2, 3, 3, 4, 6, 7 }

$$\begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \\ \wedge & \wedge & \wedge & \wedge & \wedge & \times \\ 2 & 3 & 3 & 4 & 6 & 7 \end{array}$$

$$3 + 7 \Rightarrow 10 < 12$$

$$\begin{array}{c} \text{r-l} \\ (1-5) \Rightarrow 4 \end{array}$$

$2^4 \Rightarrow \text{Ans}$

4 को 3 पर fix रखने पर 2⁴ subarray valid होत

(31) { 2, 3, 3, 4, 6, 7 }

$$\begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \\ \wedge & & & & & \times \\ 2 & 3 & 3 & 4 & 6 & 7 \end{array}$$

$$3 + 7 = 10 < 12$$

$$\begin{array}{c} \text{r-l} \\ (5-2) \Rightarrow 3 \end{array}$$

$2^3 \Rightarrow \text{Ans}$

(42) { 2, 3, 3, 4, 6, 7 }

$$\begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \\ \wedge & & & & \times & \\ 2 & 3 & 3 & 4 & 6 & 7 \end{array}$$

$$4 + 7 = 11 < 12$$

$$\begin{array}{c} \text{r-l} \\ 5-3 \Rightarrow 2 \end{array}$$

$2^2 \Rightarrow \text{Ans}$

(53) { 2, 3, 3, 4, 6, 7 }

$$\begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \\ & & & & \times & \\ 2 & 3 & 3 & 4 & 6 & 7 \end{array}$$

$$6 + 7 \Rightarrow 13 > 12$$

r-l

(64) { 2, 3, 3, 4, 6, 7 }

$$\begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \\ & & & & \times & \\ 2 & 3 & 3 & 4 & 6 & 7 \end{array}$$

$$6 + 6 < 12$$

$$\begin{array}{c} \text{r-l} \\ 5-4 \Rightarrow 0 \end{array}$$

$2^0 \Rightarrow \text{Ans}$

// Code:-

```
public int numSubseq(int[] nums, int target) {
```

```
    int mod = 1000000007;
```

```
    int n = nums.length;
```

```
    Arrays.sort(nums);
```

```
    int[] power = new int[n];
```

```
    power[0] = 1;
```

```
    for (int i = 1; i < n; i++) {
```

```
        power[i] = (power[i-1] * 2) % mod;
```

```
    }
```

```
    int left = 0;
```

```
    int right = n-1;
```

```
    int result = 0;
```

```
    while (left <= right) {
```

```
        int diff = right - left;
```

```
        if (nums[left] + nums[right] <= target) {
```

```
            result = (result + power[diff]) % mod;
```

```
            left++;
```

```
        } else {
```

```
            right--;
```

```
        }
```

```
    }
```

```
    return result;
```

```
}
```

05. Minimize Maximum Pair Sum In Array -

Input :- $\{3, 5, 2, 3\}$ output - 7

Intuition :- $\{3, 5, 2, 3\}$

$$\begin{aligned} &\{ \{3, 3\} \{5, 2\} \} \Rightarrow \{6, 7\} \Rightarrow 7 \\ &\{ \{5, 3\} \{3, 2\} \} \Rightarrow \{8, 5\} \Rightarrow 8 \end{aligned}$$

① $\{5, \underline{\quad}\}$

↓
min
↓
7

5 के साथ smallest element रखना होगा क्योंकि highest no. is our problem which will return highest val

↓ Sort

② $\{2, 3, 3, 5\}$
↑ ↑ ↓ ↓

$$\{ \{2, 5\} \{3, 3\} \} \Rightarrow \{7, 6\} \Rightarrow 7$$

Ans.

TC- $O(N \log N) + O(N)$

SC- $O(1)$

Date / /
Intimate

Code: —

```
public int minPairSum (int[] nums) {  
    int n = nums.length;
```

```
    Arrays.sort(nums);
```

```
    int i = 0;
```

```
    int j = n-1;
```

```
    int result = 0;
```

```
    while (i < j) {
```

```
        int sum = nums[i] + nums[j];
```

```
        result = Math.max(result, sum);
```

```
        i++;
```

```
        j--;
```

```
    }
```

```
    return result;
```

```
}
```

Similar to Previous Qs -

=> Boats to Save People -

Input :- people = [1, 2], Limit = 3

Output :- 1

Dry Run

Thought Process

- अगर Boat में limited people बरतते हैं तो why don't we pair the heaviest man with the lightest ~~weight~~ weight man तो हमें पता चल जाएगा की Heavy वही हलाना अच्छा जाएगा या pair में।
तो sort कर देते हैं।

{ 1, 1, 2, 3, 4, 5, 9 } Limit = 6

1+9 > 6 X boat 1

1+5 <= 6 ✓ boat 2

2+4 <= 6 ✓ boat 3

3+3 <= 6 ✓ bot 4 ✓

Code: -

```
public int numRescueBoats (int[] people, int limit) {  
    int n = people.length;  
  
    Arrays.sort (people);  
  
    int left = 0;  
    int right = n-1;  
    int boatCnt = 0;  
  
    while (left <= right) {  
        int sum = people [left] + people [right];  
  
        if (sum <= limit) {  
            boatCnt++;  
            left++;  
            right--;  
        }  
        else {  
            boatCnt++;  
            right--;  
        }  
    }  
    return boatCnt;  
}
```

3

Weekly Contest - 375

Date / /
Intimate

06. Count the Number of Good Partitions -
No Duplicates Should be here :-

Input :- nums = [1, 2, 1, 3]

Output :- 2

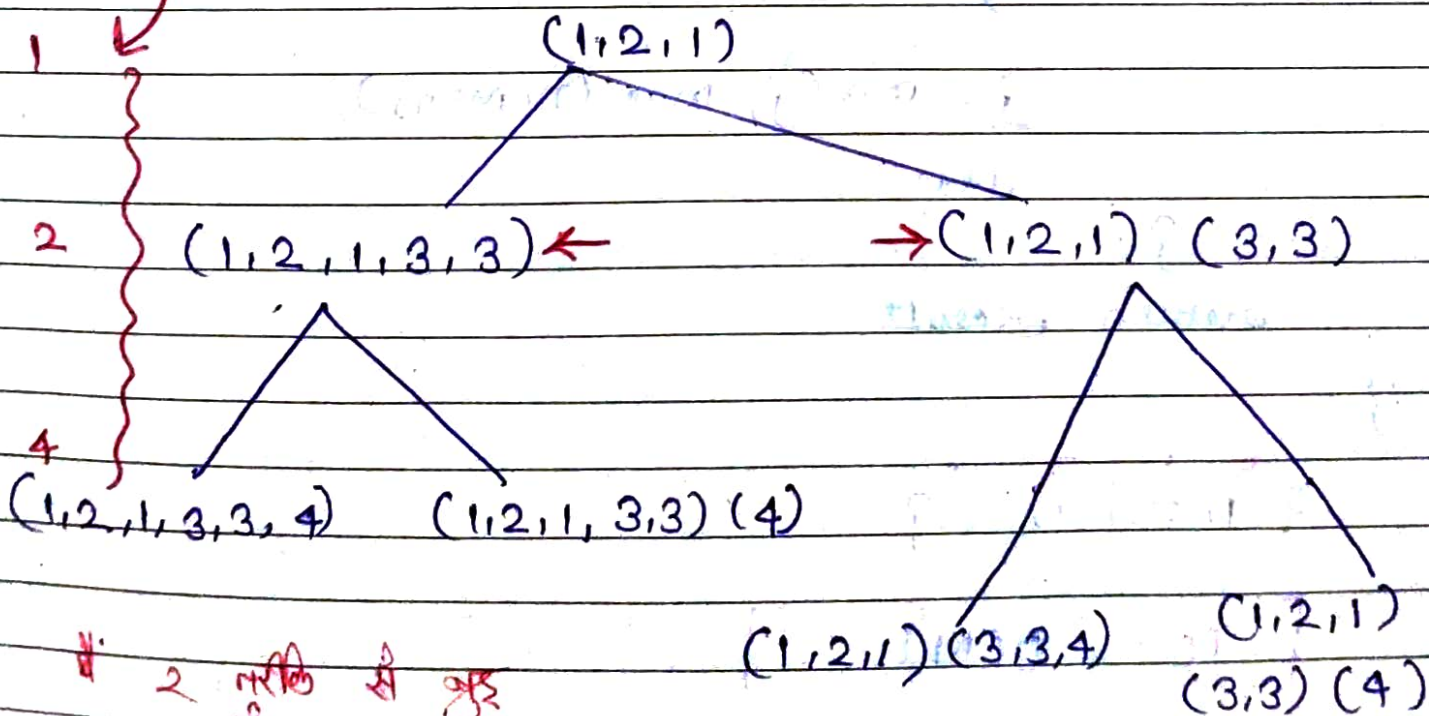
Explanation :- [(1, 2, 1) (3)]
 unique elements unique elements

⇒ DRY - RUN :- [1, 2, 1, 2, 3, 4]

① Partition Divider

$i > j$

possibilities



2 possibilities
at each level

① { 1, 2, 1, 3 }
u j

last index ↑

map
1 → 2

~~j = map[nums[i]]~~

2 → 1

Pseudo Code:-

i = 0;

j = max(j, map[nums[i]]);

while (i < n) {

if (i > j) {

result = (result * 2);
}

j = max(j, map[nums[i]]);

i++;

return result;

}

{ 1, 2, 1, 3 }

②

TC - O(N)
SC - O(N)

// Code: -

```
public int numberOfGoodPartitions (int nums[]) {
    int n = nums.length;
    int M = 1000000007;

    Map< Integer, Integer> map = new HashMap<>();

    for (int i = 0; i < n; i++) {
        map.put (nums[i], i);
    }

    int i = 0;
    int j = 0;
    j = Math.max (j, map.get (nums[0]));
    int result = 1;

    while (i < n) {
        if (i > j) {
            result = (result * 2) % M;
        }

        j = Math.max (j, map.get (nums[i]));
        i++;
    }

    return result;
}
```

07. Minimum Length of String After Deleting Similar Ends -

Example: - S = "ca"

Output - 2

Ex, S = "cabaabac"

Output 0

① ~~cabaabac~~

② ~~abaab~~

③ ~~baab~~

④ ~~ca~~ $\Rightarrow$ 0 length

Ex, S = "aabccabbac"

Output - 3

① ~~aabccabbac~~

② ~~bccabb~~

③ ~~ca~~ $\Rightarrow$ length $\Rightarrow$ 3
different

Ther Problem clearly hints at deleting from two ends $\rightarrow$ Two pointer

DRY RUN:-

" a a b c c a b b a "

$$j - i + 1 \Rightarrow 5 - 3 + 1 \Rightarrow 3$$

pseudo Code -

TC - $O(N)$
SC - $O(1)$

while ($i < j$ && $nums[i] == nums[j]$) {

char ch = $nums[i]$;

while ($i < j$ && $nums[i] == ch$) {

$i++$;

}

ex = "aqa" {corner case}

$j \geq i$

while ($j > i$ && $nums[j] == ch$) {

$j--$;

}

}

return $j - i + 1$;

}

// Code:-

```
public int minimumLength (String s) {  
    int n = s.length();
```

```
    int left = 0;
```

```
    int right = n-1;
```

```
    while (left < right && s.charAt(left) ==  
           s.charAt(right)) {
```

```
        while (left < right && s  
               .charAt(left) ==  
               s.charAt(right)) {
```

```
            while (left < right && s.charAt(left) == ch) {  
                left++;  
            }
```

```
            while (left <= right && s.charAt(right) == ch) {  
                right--;  
            }
```

```
        }  
        return right - left + 1;
```

```
    }
```

~~TC = O(2N)~~
~~SC = O(N)~~

08. Maximum Width Ramp -

Example:- `nums = [6, 0, 8, 2, 1, 5]`

Output - 4

⇒ Brute force Approach -

```

ramp = 0;
for (int i = 0 → n) {
    for (int j = i + 1 → n) {
        if (nums[i] <= nums[j]) {
            ramp = Math.max(ramp, j - i);
        }
    }
}
return ramp;
    
```

TC - $O(N^2)$
SC - $O(1)$

⇒ 2-pointers -

`[ 6, 0, 8, 2, 1, 5 ]`

TC - $O(2N)$
SC - $O(1)$

`maxToRight = [ 8, 8, 8, 5, 5, 5 ]`

`i = 0;`
↓
`nums`

`j = 0;`
↓
`maxToRight`

// Code: - (Two Pointers) -

```
public int maxWidthRamp (int nums[]) {  
    int n = nums.length;  
  
    int rightMax[] = new int[n];  
    rightMax[n-1] = nums[n-1];  
    for (int i = n-2; i >= 0; i--) {  
        rightMax[i] = Math.max(rightMax[i+1], nums[i]);  
    }
```

```
    int i = 0;  
    int j = 0;  
    int ramp = 0;
```

```
    while (j < n) {
```

Can be replaced with while $(i < j \text{ \& \& } \text{nums}[i] > \text{rightMax}[j])$ {

```
        i++;
```

if

```
        ramp = Math.max(ramp, j - i);
```

```
        j++;
```

```
    }
```

```
    return ramp;
```

```
}
```

⇒ Using Monotonic Stack

DRY RUN:-

Ex. - nums = [6, 0, 8, 2, 1, 5]

Output = 4

Thought Process

HOW TO IDENTIFY - • Involves Elements that need to be processed in certain order.

Example - Next Greater Element, Next Smaller Element

• If maintaining an order increasing / decreasing helps to reduce time complexity.

* • When comparing with all elements can be ignored to improve time complexity.

Monotonic → Increasing / Decreasing

09. Partition Labels - {Amazon, Meta}

Ex. Input: $S = \text{"ababcbacade fegdehijhklj"}$

Output: -

① We need to have information about last occurrence of char.

map.

a → 8

b → 5

c → 7

d → 14

e → 15

g → 13

f → 11

h → 19

i → 22

j → 23

k → 20

l → 21

OCN)

TC - ~~$O(2N)$~~

SC - $O(1)$

"ababcbacade fegdehijhklj"

$j + i + 1$

Dry Run

"^{0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23}
a b a b c b a c a d e f e g d e h i j h k l j"

• Store last occurrences of each character in map

a → 8
b → 5
c → 7
d → 14
e → 15
g → 13
f → 11
h → 19
i → 22
j → 23
k → 20
l → 21

// Code:- Dry Run:-

① a b a b c b a c a d e f e g d e h i j h k l i j
 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23

~~j j j j j j j~~ end

a end = (8, 8) $\Rightarrow$ 8

b end = (8, 5) $\Rightarrow$ 8

a end = (8, 8) $\Rightarrow$ 8

b end = (8, 3) $\Rightarrow$ 8

c end = (8, 7) $\Rightarrow$ 8

b end = (8, 5) $\Rightarrow$ 8

a end = (8, 8) $\Rightarrow$ 8

c end = (8, 7) $\Rightarrow$ 8

~~a end $\Rightarrow$ (8, 8) $\Rightarrow$ 8~~

result = 8 - 0 $\Rightarrow$ 8 + 1 $\Rightarrow$ 9

result = {9}

② a b a b c b a c a d e f e g d e h i j h k l i j
 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23

end = 14

a end $\rightarrow$ (14, 15) $\Rightarrow$ 15

f end $\rightarrow$ (15, 11) $\Rightarrow$ 15

e end $\rightarrow$ (15, 15) $\Rightarrow$ 15

g end $\rightarrow$ (15, 13) $\Rightarrow$ 15

d end $\rightarrow$ (15, 14) $\Rightarrow$ 15

~~a end $\rightarrow$ (15, 15) $\Rightarrow$ 15~~ jef

result $\Rightarrow$ {9, 7}

//Code:-

```

public List<Integer> partitionLabels (String s) {
    int n = s.length();
    List<Integer> result = new ArrayList<>();

    int mp[] = new int[26];
    for (int i = 0; i < n; i++) {
        mp[s.charAt(i) - 'a'] = i;
    }

    int i = 0;
    while (i < n) {
        int end = mp[s.charAt(i) - 'a'];
        int j = i;

        while (j < end) {
            end = Math.max(end, mp[s.charAt(j) - 'a']);
            j++;
        }

        result.add(j - i + 1);
        i = j + 1;
    }

    return result;
}

```

10. Push Dominoes -

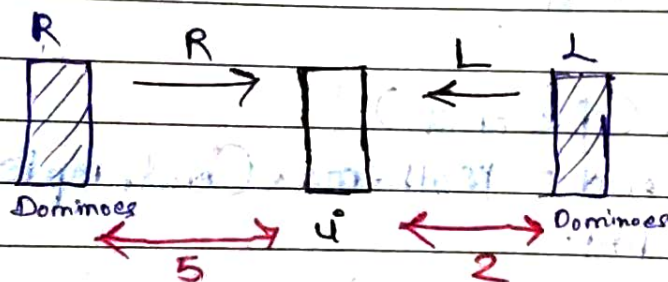
Example: - dominoes = "RR.L"

Output: - "RR.L"

Ex dominoes = ".L.R...LR..L.."

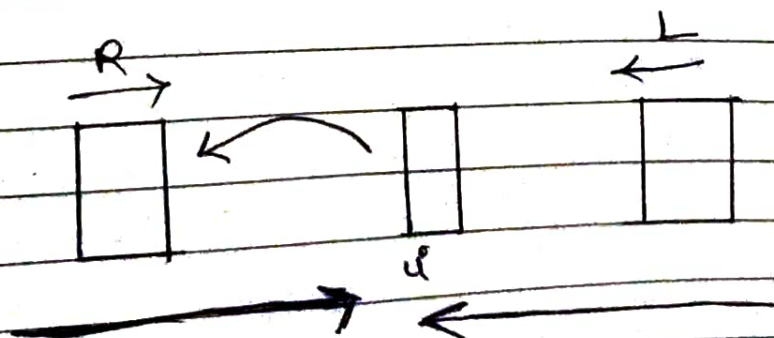
Output - "LL.RR.LLRLL.."

Thought Process

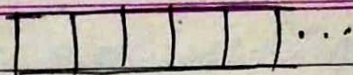


If left side distance is small, then force will be calculated of left side,
• Same for right side condition

Important Note: -



"leftClosestR"



"rightClosestL"



Left Hand Side

मे से
किसी

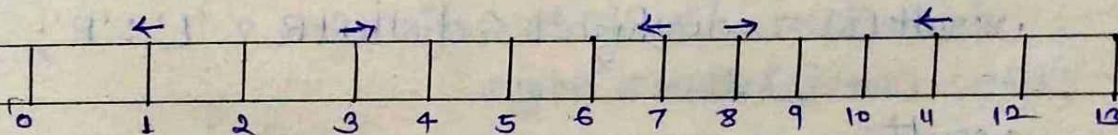
Right side ~~push~~ ^{force} कितना आ रहा है
उंth domino के लिए

Right Hand Side

मे से
किसी

left side force कितना आ रहा है
उंth domino के लिए ...

• L • R • • • L R • • L • •



left
Closest
R

0	1	2	3	4	5	6	7	8	9	10	11	12	13
-1	-1	-1	3	3	3	3	-1	8	8	8	-1	-1	-1

Left to Right

right
Closest
L

0	1	2	3	4	5	6	7	8	9	10	11	12	13
1	1	-1	-1	7	7	7	7	-1	11	11	11	-1	-1

Right to Left

0	1	2	3	4	5	6	7	8	9	10	11	12	13
L	L	•	R	R	•	L	L	R	R	L	L	•	•

$$\text{distLeftR} = \text{abs}(i - \text{leftClosestR}(i));$$

$$\text{distRightL} = \text{abs}(i - \text{rightClosestL}(i));$$

$$4 - 3 = 1$$

$$4 - 7 = 3$$

Story points -

```

if (weightClosestL == weightClosestR) {
    result = '.';
} else if (weightClosestL == -1) {
    result = 'R';
} else if (weightClosestR == -1) {
    result = 'L';
} else if (distR == distL) {
    result = '.';
} else {
    result = distRight < distLeft ? 'L' : 'R';
}
return result;

```

// Code:-

```

public String pushDominoes (String dominoes) {
    int n = dominoes.length();

    int[] rightClosestL = new int[n];
    int[] leftClosestR = new int[n];

    for (int i = n-1; i >= 0; i--) {
        char ch = dominoes.charAt(i);
        if (ch == 'L') {
            rightClosestL[i] = i;
        } else if (ch == '.') {
            rightClosestL[i] = (i < n-1) ?
                rightClosestL[i+1] : -1;
        } else {
            rightClosestL[i] = -1;
        }
    }

    for (int i = 0; i < n; i++) {
        char ch = dominoes.charAt(i);
        if (ch == 'R') {
            leftClosestR[i] = i;
        } else if (ch == '.') {
            leftClosestR[i] = (i > 0) ?
                leftClosestR[i-1] : -1;
        } else {
            leftClosestR[i] = -1;
        }
    }
}

```

```
StringBuilder result = new StringBuilder();
```

```
for (int i = 0; i < n; i++) {
    int distRightL = (rightClosestL[i] == -1) ?
```

```
Integer.MAX_VALUE;
```

```
Math.abs(i - rightClosestL[i]);
```

```
int distLeftR = (leftClosestR[i] == -1) ?
```

```
Integer.MAX_VALUE;
```

```
Math.abs(i - leftClosestR[i]);
```

can
be

removed

```
if (rightClosestL[i] == leftClosestR[i]) {
```

```
    result.append('.');
```

```
} else if (rightClosestL[i] == -1) {
```

```
    result.append('R');
```

```
} else if (leftClosestR[i] == -1) {
```

```
    result.append('L');
```

```
} else if (distLeftR == distRightL) {
```

```
    result.append('.');
```

```
} else {
```

```
    result.append(distRightL < distLeftR ? 'R' : 'L');
```

```
}
```

```
}
```

```
return result.toString();
```

```
}
```

Code // Optimal: —

```
#public String pushDominoes (String dominoes) {  
    char[] arr = dominoes.toCharArray();  
    int n = arr.length;  
    int prev = -1;  
    char prevChar = 'L'  
  
    for (int i=0; i<=n; i++) {  
        char curChar = (i==n) ? 'R' : arr[i];  
  
        if (curChar == '.') continue;  
  
        if (curChar == prevChar) {  
            for (int k = prev+1; k<i; k++) {  
                arr[k] = prevChar;  
            }  
        } else if (prevChar == 'R' && curChar == 'L') {  
            int left = prev+1, right = i-1;  
  
            while (left < right) {  
                arr[left] = 'R';  
                arr[right] = 'L';  
                left++;  
                right--;  
            }  
            prev = i;  
            prevChar = curChar;  
        }  
  
        return new String (arr);  
    }  
}
```

11. Find the Lexicographically Largest String from the Box I -

Example 01: - Input: - word = "dbca", numFriends = 2

Output: - "dbc"

Example 02 - Input - word = "gggg", numFriends = 4

Output - "g"

— Thought Process —

Lexicographically largest - which appears later in the dictionary.

① word = "dbca", numFriends = 2

Look for largest character → 'd'

Friend1

Friend2

d

bca

db

ca

dbc

a

Friends1

d

db

Friends2

bc

c

Friends3

a

a

$$\Rightarrow n - (\text{numFriends} - 1)$$

$$n - \frac{1}{2}$$

Ex.

" a b c e d "

F=3

F1

F2

F3

→ 3 ch

1 char

1 char

" ed "

3 characters नहीं मिल पाये, un substr
में 2 की दे सकते हैं।
(max-3 दे सकते हैं)

Ex.

" e a b c e d "

F=3

" e g b c " < " e d "

Can take length $\Rightarrow$

क्योंकि हो सकता है कम elements present हो।

// Psuedo Code —

longestpossible = n - (numstr - 1);

for (int i = 0; i < n; i++) { → O(N)

 cantakelength = min (largestpossible, n - i);

i + cantakelength

 result = max (result, word.substr(i, ~~cantakelength~~ i + cantakelength));

}

→ O(N)

return result;

TC -	$O(N^2)$
SC -	$O(1)$

1 Find को सारे character दे दो After giving
1 char each to ~~(n-1)~~ (find-1) friends.

⇒ e a b c e d

1/ Brute code: — ✓

```
public String answerString (String word, int numFriends) {
    int n = word.length();
    if ( numFriends == 1 ) {
        return word;
    }
```

```
    String result = "";
```

```
    int longestPossible = n - (numFriends - 1);
```

```
    for ( int i = 0; i < n; i++ ) {
        int canTakeLength = Math.min (longestPossible, n - i);
```

```
        String curr = word.substring (i, i + canTakeLength);
```

```
        if ( curr.compareTo (result) > 0 ) {
            result = curr;
        }
```

```
    }
```

```
    return result;
```

```
}
```

Better :-

" e a b c e d c f e z " , friends = 3

~~k~~ = 1

e > a $\Rightarrow$ j++

e > b $\Rightarrow$ j++

e > c $\Rightarrow$ j++

" e a b c e d c f e z " , f = 3

e == e j++ j++

a < d $\Rightarrow$ i == j j++

" e a b c e d c f e z " , f = 3

e > c $\Rightarrow$ j++

e < f $\Rightarrow$ ~~j++~~ ~~k++~~ i == j, j++

" e a b c e d c f e z " , f = 3

f > e j++

z > f i == j $\Rightarrow$ j++

" e a b c e d c f e z "

ans = z

index = i { Lexicographically largest substring will start from index i }

// Code: - Two Pointer Approach -

```

public String int lastSubstring (String s) {
    best starting point
    int i = 0;
    int j = 1;
    int n = s.length();

    while (j < n) {
        int k = 0;
        // skipping equal characters
        while (j+k < n && s.charAt(i+k) == s.charAt(j+k)) {
            k++;
        }

        if (j+k < n && s.charAt(i+k) < s.charAt(j+k)) {
            int t = i;
            i = j;
            j = Math.max(j+1, t+k+1); j+1;
        } else {
            j = j+k+1;
        }
    }

    return s.substring(i, j) (i);
}
    
```

```
public String answerString (String word,  
    int numFriends) {
```

```
    if ( numFriends == 1 ) {  
        return word;  
    }
```

```
    int String last = lastSubstring (word);
```

```
    int n = word.length();
```

```
    int longestPossibleLength = n - (numFriends - 1);
```

```
    int m = last.length();
```

```
    int canTakePossible = min (longestPossibleLength, n - 1);
```

```
    return last.substring (0, Math.min (m,
```

```
        n - numFriends + 1));
```

```
    return word.substring (i, canTakePossible  
        + i);
```

```
}
```

12. K-diff pairs in an Array -

Ex. Input - nums = [3, 1, 4, 1, 5], k = 2

Output - 2

Explanation - There are two 2-diff pairs in the array, (1, 3) and (3, 5).

DRY RUN USING HashMap -

" 3 1 4 1 5 "

k = 2

① put all elements into hashmap with its frequency

② Traverse over the map and check

5	-1
4	-1
1	-2
3	-1

if $k = 0$ then if same element exist ~~as~~, return freq. because diff 0 means same element

③ if ~~k~~ is other than 0 then

11 Code (Using HashMap) -

```
public int findPairs ( int nums[], int k) {  
    int n = nums.length;  
    int cnt = 0;  
  
    Map < Integer, Integer> map = new HashMap <>();  
  
    for (int i = 0; i < n; i++) { → O(N)  
        map.put (nums[i], map.getOrDefault (nums[i],  
            0) + 1);  
    }  
  
    for (int key : map.keySet()) { → O(N)  
        if (k == 0) {  
            if (map.get (key) > 1) {  
                cnt++;  
            }  
        } else if (map.containsKey (key + k)) {  
            cnt++;  
        }  
    }  
  
    return cnt;  
}
```

Time - $O(2N)$
Space - $O(N)$

DRY RUN USING Two pointers -

```
public int findPairs (int nums[], int k) {
    int n = nums.length;
```

```
    Arrays.sort(nums);
```

```
    int left = 0;
```

```
    int right = 1;
```

```
    int cnt = 0;
```

```
    while (right < n) {
```

```
        if (left == right || nums[right] - nums[left] < k) {
```

```
            right++;
        }
```

```
        else if (nums[right] - nums[left] > k) {
```

```
            left++;
        }
```

```
        else {
```

```
            cnt++;
```

```
            left++;
```

```
            right++;
        }
```

```
    while (left < n && nums[left] == nums[left-1])
```

```
        left++;
```

```
    }
```

```
    while (right < n
```

```
        && nums[right] ==
```

```
        nums[right + 1]) {
```

```
            right++;
```

```
    }
```

```
    return cnt;
```

1, 3, 1, 5, 4

Code (Using Two Pointers) -

$k = 1$

① Sorted: "1 3 3 4 5"
l u

$3 - 1 \Rightarrow 2$

~~left++~~ cnt = 1
right++
left++

② "1 3 3 4 5"
l r

$k = 1$

$3 - 1 \Rightarrow 2 > k$ left++
~~left++~~ ~~right++~~

③ "1 3 4 5"
l r

$k = 1$

$l = r$
 $4 - 3 = 1$ right++

1 1 3 4 5
l r

$4 - 3 = 1$

① 1 1 3 4 5 $k = 1$
l r

$r - l = 0 < k$ right++

② 1 1 3 4 5 $k = 1$
l r

$3 - 1 = 2 > k$ left++

③ 1 1 3 4 5 $k = 1$
l r

$l = r$ right++

Date / /
Intimate

Ex: Input : version1 = "1.2", version2 = "1.10"

Output: -1

① $y=0$

(2) $i^0 = 1$. $2 < 10$

v_2, γ, v_1

rotation - 1

$$u_1 = 2, 0, 1, 10 \quad u_2 = 2, 0, 1, 0$$

$$[2, 0, 1, 10] \quad [2, 0, 1, 0]$$

① $i=1$ $2 \neq 1 \quad 2$

② 0 7 0

$$v_2 > v_1$$

return -1

Code: -

```
public int compareVersion(String version1,
                           String version2) {
    String[] v1 = version1.split("\\.");
    String[] v2 = version2.split("\\.");

    for (int i = 0; i < Math.max(v1.length,
                                   v2.length); i++) {

        int num1 = i < v1.length ?
                    Integer.parseInt(v1[i]) : 0;

        int num2 = i < v2.length ?
                    Integer.parseInt(v2[i]) : 0;

        if (num1 < num2) {
            return -1;
        }

        if (num1 > num2) {
            return 1;
        }
    }

    return 0;
}
```

Return Statement as per Qs.

11 Dry Run 2 pointers —

$v1 = "2.0.1.10"$

$v2 = "2.01"$

① $i=0$ $num1 = 0 \times 10 + 2 \Rightarrow 2$

$i++$
 $j=0$ $num2 = 0 \times 10 + 2 \Rightarrow 2$

② $num1 == num2$ $i++$ $j++$

③ $i=2$ $num1 = 2 \times 10 + 0 \Rightarrow 20$

$i++$

$j=2$ $num2 = 2 \times 10 + 0 \Rightarrow 20$

$j++$

$j=3$

$num2 = 20 \times 10 + 1 \Rightarrow 201$

$j++$

$num2 > num1$

return (-1)

Ans

//code - two pointers -

```
public int compareVersion (String version1,  
                           String version2) {
```

```
    int n = version1.length();
```

```
    int m = version2.length();
```

```
    int i = 0;
```

```
    int j = 0;
```

```
    while (i < n || j < m) {
```

```
        int num1 = 0;
```

```
        while (i < n && version1.charAt(i) != '.') {
```

```
            num1 = num1 * 10 +
```

```
                (version1.charAt(i) - '0');
```

```
            i++;
```

```
        int num2 = 0;
```

```
        while (j < m && version2.charAt(j) != '.') {
```

```
            num2 = num2 * 10 +
```

```
                (version2.charAt(j) - '0');
```

```
            j++;
```

```
        if (num1 > num2) {
```

```
            return 1;
```

```
        } else if (num1 < num2) {
```

```
            return -1;
```

```
        }
```

```
        i++;
```

```
        j++;
```

```
    }
```

```
    return 0;
```

```
}
```

14. Count the Number Of Fair Pairs -

Input:- nums = [0, 1, 7, 4, 4, 5], lower = 3, upper = 6

Output:- 6

Brute Force

ans = 0

for (int i = 0 → n) {

for (int j = i + 1 → n) {

int Sum = nums[i] + nums[j];

if (Sum ≥ lower && Sum ≤ upper) {
ans++;

}

}
return ans;

Optimal

Hint:- For each number in the array, keep track of the smallest and largest numbers in the array that can form a fair pair with this number.

You can use Lower Bound & Upper Bound Concept

Ex. [0, 1, 4, 4, 5, 7] ⇒ sorted...

for 0 you will need to find the smallest which satisfies $\geq$ lower condition

low ⇒ (nums[i] - lower) abs

Same goes for upper Bound

Code:- II Binary Search -

```
public long countFairPairs (int nums[], int lower,
                             int upper) {
```

```
    int n = nums.length;
```

```
    Arrays.sort(nums);
```

```
    long ans = 0;
```

```
    for (int i = 0; i < n; i++) {
```

```
        int minReq = lower - nums[i];
```

```
        int maxReq = upper - nums[i];
```

```
        int low = lowerBound (nums, i+1,
                               nums.length-1, minReq);
```

```
        int up = upperBound (nums, i+1,
                              nums.length-1, maxReq);
```

```
        ans += (up - low);
```

```
    }
```

```
    return ans;
```

```
}

private int lowerBound (int nums[], int start,
                        int end, int target) {
```

```
    int low = start;
```

```
    int high = end;
```

```
    while (low <= high) {
```

```
        int mid = low + (high - low) / 2;
```

```
        if (nums[mid] >= target) {
```

```
            high = mid - 1;
```

```
        } else {
```

```
            low = mid + 1;
```

```
        }

    }

    return low;
```

```
private int upperBound (int nums[], int start,  
    int end, int target) {  
    int low = start;  
    int high = end;
```

```
    while (low <= high) {  
        int mid = low + (high - low) / 2;
```

```
        if (nums[mid] > target) {  
            high = mid - 1;
```

```
        }  
        else {  
            low = mid + 1;
```

```
        }  
    }
```

```
    return low;
```

```
}
```

15. Happy Number -

Input :- $n = 19$

Output :- true

Explain - $1^2 + 9^2 = 82$
 $8^2 + 2^2 = 68$
 $6^2 + 8^2 \Rightarrow 100$
 $1^2 + 0^2 + 0^2 \Rightarrow 1 \checkmark$

— DRY RUN —

61

$$6^2 + 1^2 = 37$$

$$3^2 + 7^2 = 58$$

$$5^2 + 8^2 = 89$$

$$8^2 + 9^2 = 145$$

$$1^2 + 4^2 + 5^2 = 42$$

$$4^2 + 2^2 = 20$$

$$2^2 + 0^2 = 4$$

$$4^2 = 16$$

$$1^2 + 6^2 = 37$$

same

Stop

false

cycle

// Using Set

Date / /
Intimate

```
public boolean isHappy (int n) {  
    Set < Integer > visit = new HashSet<>();  
  
    while ( ! visit.contains(n) ) {  
        visit.add (n);  
  
        n = getNextNumber(n);  
  
        if (n == 1) {  
            return true;  
        }  
    }  
    return false;  
}
```

```
private int getNextNumber (int n) {  
    int output = 0;  
  
    while ( n > 0 ) {  
        int digit = n % 10;  
        output += digit * digit;  
  
        n = n / 10;  
    }  
    return output;  
}
```

Using 2 pointers -

```

public boolean isHappy (int n) {
    int slow = getNextNumber(n);
    int fast = getNextNumber(getNextNumber(n));

    while (slow != fast) {
        if (fast == 1) {
            return true;
        }
        slow = getNextNumber(slow);
        fast = getNextNumber(getNextNumber(fast));
    }
    return slow == 1;
}

private int getNextNumber(int n) {
    int output = 0;

    while (n > 0) {
        int digit = n % 10;
        output += digit * digit;
        n = n / 10;
    }

    return output;
}
    
```